

Proyecto Final - Arquitectura Orientada a Microservicios

Contexto del Negocio

Su equipo ha sido contratado para diseñar y desarrollar un **sistema de gestión de empleados** para una empresa en crecimiento. El departamento de **Recursos Humanos (RRHH)** necesita automatizar y modernizar sus procesos críticos relacionados con el ciclo de vida del empleado, desde su incorporación hasta su salida de la organización.

Actualmente, todos estos procesos se realizan manualmente, lo que genera:

- Retrasos** en la creación de credenciales para nuevos empleados

- Inconsistencias** en la gestión de accesos

- Falta de visibilidad** sobre el estado de las cuentas

- Problemas de seguridad** por cuentas activas de empleados que ya no laboran en la empresa

- Dificultad** para notificar cambios importantes a los empleados

La empresa requiere una solución moderna, escalable y mantenible basada en arquitectura de microservicios que permita automatizar estos procesos críticos.

Descripción del Sistema

Deberá construir un sistema con **arquitectura basada en microservicios** que soporte el siguiente ciclo de vida del empleado:

1. Onboarding (Incorporación)

Cuando RRHH registra un nuevo empleado en el sistema:

- Se deben crear automáticamente las credenciales de acceso (usuario y contraseña) Se debe enviar una notificación por correo electrónico al empleado con sus credenciales y pasos de activación

- El empleado debe poder acceder al sistema y gestionar su perfil personal

2. Gestión de Perfil

Los empleados autenticados deben poder:

- Actualizar información personal (nombre, apellido, teléfono)
- Actualizar información de contacto (dirección, ciudad, código postal)
- Actualizar información profesional (cargo, área, número de empleado)
- Ver su información de manera centralizada

3. Gestión de Vacaciones

Cuando RRHH programa vacaciones para un empleado:

La cuenta del empleado se desactiva automáticamente en la fecha de inicio de las vacaciones La cuenta se reactiva automáticamente cuando el empleado regresa (fecha de fin de vacaciones) Se envía una notificación al empleado confirmando el período de vacaciones

4. Offboarding (Salida)

Cuando un empleado es retirado de la empresa:

- RRHH marca al empleado como "inactivo" o "retirado"
- Todas las credenciales y accesos del empleado se desactivan permanentemente
- Se genera un registro de auditoría de la fecha y hora de la desactivación

5. Observabilidad

El sistema debe proveer mecanismos de observabilidad que permitan:

- Logs centralizados:** Todos los microservicios deben enviar logs a un sistema central
- Monitoreo:** Visualización del estado y salud de cada microservicio
- Alertas:** Notificaciones cuando ocurran errores críticos o comportamientos anómalos

Microservicios Requeridos

Su solución debe incluir al menos los siguientes microservicios (pueden ser propios o de terceros):

Microservicios Principales

1. Microservicio de Gestión de Empleados

CRUD completo de empleados (crear, leer, actualizar, eliminar)

Estados del empleado: ACTIVO , EN_VACACIONES , RETIRADO

Publicar eventos cuando se crea, actualiza o elimina un empleado

Campos sugeridos: nombre, apellido, email, número de empleado, fecha de ingreso, cargo, área, estado

2. Microservicio de Autenticación y Autorización

Registro de credenciales (usuario/contraseña)

Login con generación de tokens (JWT u otro mecanismo)

Validación de tokens

Activación/desactivación de cuentas

Escuchar eventos de empleados para activar/desactivar cuentas automáticamente **Importante:**

Este microservicio NO debe exponer un endpoint público de registro (/register). Las credenciales se crean **automáticamente de forma asíncrona** al consumir el evento empleado.creado . Solo el proceso interno event-driven puede crear cuentas, no usuarios externos.

3. Microservicio de Gestión de Perfiles

Gestión del perfil personal del empleado

Cada empleado autenticado puede actualizar su propio perfil

Información complementaria: foto de perfil, biografía, redes sociales, dirección completa

Sincronización con el microservicio de empleados

4. Microservicio de Gestión de Vacaciones

CRUD de períodos de vacaciones

Programación de vacaciones (fecha inicio, fecha fin)

Publicar eventos para activar/desactivar cuentas según fechas

Validaciones: no solapamiento de períodos, días disponibles, etc.

5. Microservicio de Notificaciones

Envío de correos electrónicos

Escuchar eventos del sistema (nuevo empleado, vacaciones, desactivación)

Plantillas de notificación configurables

Eventos a escuchar:

- Empleado creado → enviar credenciales
- Vacaciones programadas → enviar confirmación
- Cuenta desactivada → enviar notificación de salida

6. API Gateway

- Punto de entrada único para todas las peticiones del cliente
- Enrutamiento de peticiones a los microservicios correspondientes
- Validación de tokens JWT
- Composición de respuestas (ej: combinar datos de empleado + perfil)

Nota: No expone endpoint de registro público. Las credenciales se crean automáticamente vía eventos cuando RRHH registra un empleado

Operaciones expuestas:

- POST /auth/login - autenticación
- POST /auth/change-password - cambio de contraseña (usuario autenticado)
- GET /employees - listar empleados
- POST /employees - crear empleado
- GET /employees/{id} - obtener empleado completo (datos + perfil)
- PUT /employees/{id} - actualizar empleado
- DELETE /employees/{id} - eliminar empleado
- GET /profile - obtener perfil del empleado autenticado
- PUT /profile - actualizar perfil
- POST /vacations - programar vacaciones
- GET /vacations - consultar vacaciones

Microservicios de Observabilidad

7. Logs Centralizados

Implementar o integrar una solución de centralización de logs (ej: ELK Stack, Loki, Graylog)

- Todos los microservicios deben enviar logs estructurados
- Logs deben incluir: timestamp, nivel, servicio, mensaje, contexto

8. Monitoreo

Implementar o integrar una solución de monitoreo (ej: Prometheus + Grafana, Datadog)

Métricas de salud de cada microservicio

Métricas de negocio: empleados creados, notificaciones enviadas, errores

Dashboards visuales

Requisitos Técnicos por Microservicio

Cada microservicio que desarrolle debe cumplir con:

1. Documentación API

Especificación OpenAPI (Swagger) completa

Interfaz interactiva de prueba (Swagger UI)

Documentación de modelos de datos y respuestas

2. Pruebas Automatizadas

Pruebas unitarias de lógica de negocio

Pruebas de integración que verifiquen el camino feliz de todas las operaciones

Cobertura mínima: 70%

3. Health Check

Implementar endpoint /health en todos los microservicios

Responder con estado del servicio y dependencias

Formato JSON: { "status": "UP", "dependencies": {...} }

4. Pipeline CI/CD

Pipeline de integración continua en Jenkins

Etapas mínimas: build, test, package

Generación de artefactos (imágenes Docker)

Diagrama de Pipeline CI/CD



Flujo del Pipeline:

Bases de Datos: PostgreSQL (x3) + MongoDB, cada servicio con su DB
💎💎 **Observabilidad:** Stack ELK (Elasticsearch, Logstash, Kibana) + Prometheus + Grafana
💎💎 **CI/CD:** Jenkins + Docker Registry local

Comando de inicio:

```
docker-compose up --build
```

Arquitectura del Sistema

Comunicación entre Microservicios

Los microservicios deben comunicarse mediante **mensajería asíncrona** utilizando un message broker de su elección (RabbitMQ, Apache Kafka, Redis Streams, NATS, etc.).

[!IMPORTANT]

Comunicación sincrónica (REST): Solo entre Cliente ↔ API Gateway ↔ Microservicios

Comunicación asíncrona (Events): Entre microservicios a través del message broker **NO**

exponer endpoints públicos de registro: Las credenciales se crean automáticamente vía eventos internos

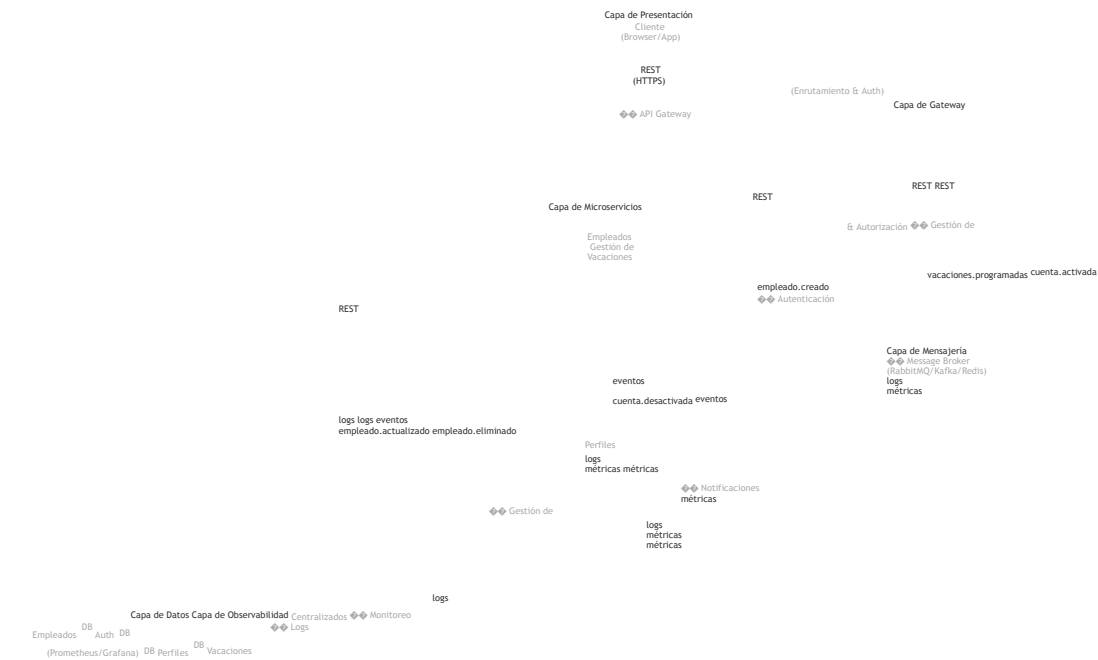
Eventos Principales

Evento	Publicado por	Consumido por	Descripción
empleado.creado	Gestión de Empleados	Autenticación, Notificaciones	Se creó un nuevo empleado
empleado.actualizado	Gestión de Empleados	Perfil	Se actualizó información del empleado
empleado.eliminado	Gestión de Empleados	Autenticación, Perfil	Se eliminó un empleado
vacaciones.programadas	Gestión de Vacaciones	Autenticación, Notificaciones	Se programaron vacaciones

cuenta.activada	Autenticación	Notificaciones	Se activó una cuenta
-----------------	---------------	----------------	----------------------

Evento	Publicado por	Consumido por	Descripción
cuenta.desactivada	Autenticación	Notificaciones	Se desactivó una cuenta

Diagrama de Arquitectura (Referencia)



Leyenda:

Líneas sólidas (→): Comunicación sincrónica REST/HTTPS

Líneas punteadas (->): Comunicación asíncrona mediante eventos

Emojis: Identificación visual rápida de cada componente

Restricciones Técnicas

[!CAUTION]

El incumplimiento de estas restricciones afectará directamente su calificación.

Restricciones Obligatorias

1. Comunicación Asíncrona

Los microservicios DEBEN comunicarse mediante un sistema de gestión de mensajes (message broker)

Comunicación sincrónica (REST) solo se permite entre API Gateway y microservicios

2. Diversidad Tecnológica

Debe emplear **al menos 4 lenguajes de programación diferentes** para el desarrollo de los microservicios

Ejemplos: Java, Python, Node.js, Go, C#, Ruby, etc.

3. Despliegue Unificado

Debe existir un archivo docker-compose.yml en la raíz del proyecto

Con **un solo comando** (docker-compose up) todo el sistema debe:

- Compilarse

- Empaquetarse

- Desplegarse

Incluir una instancia de Jenkins en el Docker Compose

4. Entregables

Código fuente comprimido (.zip o .tar.gz)

Al descomprimir debe quedar un directorio con estructura clara

Incluir archivo [README.md](#) con instrucciones de ejecución

Criterios de Evaluación

La evaluación del proyecto se realizará considerando los siguientes aspectos:

1. Funcionalidad (40%)

- ✓ Todos los microservicios implementados y funcionando
- ✓ Flujos de negocio completos: onboarding, gestión de perfil, vacaciones, offboarding
- ✓ Comunicación mediante eventos funcionando correctamente
- ✓ API Gateway enrutando correctamente las peticiones

2. Calidad Técnica (30%)

- ✓ Especificaciones OpenAPI completas y correctas
- ✓ Pruebas automatizadas implementadas y pasando
- ✓ Health checks implementados
- ✓ Dockerfiles correctos y optimizados
- ✓ Código limpio y bien estructurado

3. Observabilidad (15%)

- ✓ Logs centralizados funcionando
- ✓ Sistema de monitoreo implementado
- ✓ Dashboards con métricas relevantes
- ✓ Alertas configuradas

4. DevOps (10%)

- ✓ Pipelines de CI en Jenkins funcionando
- ✓ Docker Compose desplegando todo el sistema correctamente
- ✓ Documentación de despliegue clara

5. Documentación (5%)

- ✓ README con instrucciones claras
- ✓ Arquitectura documentada
- ✓ Guías de configuración de herramientas de terceros

Entregables

Debe entregar un archivo comprimido que contenga:

Estructura del Proyecto

```
proyecto-final-microservicios/  
|  
├── README.md # Instrucciones generales ─── docker-compose.yml #  
Orquestación completa  
├── docker-compose.dev.yml # (Opcional) Configuración para desarrollo |  
├── microservicios/  
| ─── gestion-empleados/ # Microservicio 1
```

```

| | |— Dockerfile
| | |— Jenkinsfile
| | |— src/
| | |— tests/
| | |— swagger.yaml
| |
| |— autenticacion/ # Microservicio 2
| | |— Dockerfile
| | |— Jenkinsfile
| | |— src/
| | |— tests/
| | |— openapi.yaml
| |
| |— gestion-perfiles/ # Microservicio 3
| |— gestion-vacaciones/ # Microservicio 4
| |— notificaciones/ # Microservicio 5
| |— api-gateway/ # Microservicio 6
|
|— observabilidad/
| |— logs/ # Configuración de logs centralizados | |— monitoreo/ # Configuración de monitoreo |
| |— dashboards/ # Dashboards de Grafana o similar |
| |— docs/
| |— arquitectura.md # Diagrama y descripción de arquitectura | |— eventos.md # Documentación de
eventos | |— guías/ # Guías de configuración |
| |— scripts/
| | |— init-db.sql # Scripts de inicialización
| | |— seed-data.sql # Datos de prueba

```

README.md Mínimo

El archivo [README.md](#) debe incluir:

1. Descripción del Proyecto

Resumen de la solución implementada

Arquitectura general

2. Requisitos Previos

Docker y Docker Compose instalados

Versiones mínimas requeridas

3. Instrucciones de Ejecución

```
# Clonar o descomprimir el proyecto
```

```
cd proyecto-final-microservicios
```

```
# Levantar todo el sistema  
docker-compose up --build
```

```
# Acceder a los servicios  
# - API Gateway: http://localhost:8080  
# - Jenkins: http://localhost:9090  
# - Swagger UI: http://localhost:8080/swagger # - Grafana:  
http://localhost:3000
```

4. Tecnologías Utilizadas

- Listar lenguajes de programación por microservicio
- Message broker utilizado
- Bases de datos utilizadas
- Herramientas de observabilidad

5. Endpoints Principales

- Listado de endpoints del API Gateway
- Ejemplos de uso con curl

6. Credenciales por Defecto

- Usuario administrador
- Accesos a herramientas (Grafana, Jenkins, etc.)

Recomendaciones

Gestión del Proyecto

1. Planificación

- Dividan el trabajo en sprints
- Definan interfaces de eventos primero
- Implementen el message broker temprano

2. Desarrollo Incremental

- Comiencen con 2-3 microservicios básicos
- Agreguen observabilidad rapidamente
- Integren continuamente

3. Pruebas

- Prueben cada microservicio de manera aislada
- Prueben la integración completa frecuentemente
- Automaticen las pruebas desde el inicio

Herramientas Sugeridas

Componente	Opciones Sugeridas
Message Broker	RabbitMQ, Apache Kafka, Redis Streams
Logs Centralizados	ELK Stack (Elasticsearch + Logstash + Kibana), Loki + Promtail
Monitoreo	Prometheus + Grafana
API Gateway	Kong, Nginx, Express Gateway, propio
Base de Datos	PostgreSQL, MySQL, MongoDB (según microservicio)

Lenguajes Sugeridos

Java/Spring Boot: Microservicios empresariales (empleados, autenticación)

Python/FastAPI: Microservicios de datos y procesamiento (notificaciones)

Node.js/Express: API Gateway

Go: Servicios de alto rendimiento (logs, monitoreo)

Soporte y Consultas

Para aclaraciones sobre el proyecto:

1. Revise este documento completamente
2. Consulte la documentación de las tecnologías elegidas
3. Las herramientas de terceros deben incluir guía de configuración
4. Asista a las sesiones de asesoría del curso

Equipos de trabajo

El proyecto está pensado para que sea derrallado en grupo de máximo 4 personas.

Fecha de Entrega

Semana 15

La entrega se realizará mediante **Class Room**.

Recursos Adicionales

[Documentación de Docker Compose](#)

[Especificación OpenAPI](#)

[Patrones de Microservicios](#)

[Event-Driven Architecture](#)

[The Twelve-Factor App](#)

IMPORTANTE

Este proyecto es una oportunidad para aplicar todos los conocimientos adquiridos en el curso.

Se espera profesionalismo, calidad técnica y trabajo en equipo. Buena suerte! 💎💎